Gödel's Second Incompleteness Theorem in a Guard/Rose Equational System: Formalization in Agda

1 Overview

We formalize Gödel's first and second incompleteness theorems in a purely equational system combining two ideas:

- *Guard*'s syntax [**?**]: binary trees with typed unary and binary functors, serving as both the data domain and the term language;
- *Rose*'s purely equational proof style [**?**]: the only judgement is an equation $t = u$ between terms, with no propositional connectives, no sequents, and no logical rules — only computation axioms, congruence, substitution, and a tree-induction principle (**Schema F**).

The formalization is in Agda with `--safe --without-K --exact-split`. No postulates.

Theorem 1 (Gödel I, constructive form — `godelIToBot` in `ProofEAny.agda`)**.** *For any hypothesis equation H:*

$$H \vdash G \;\rightarrow\; H \vdash \mathbf{t} = \mathbf{f}$$

where G is the Gödel sentence.

Theorem 2 (Gödel II, constructive form — `conToBot` in `GodelIIFull.agda`)**.** *For any hypothesis equation H:*

$$H \vdash \mathrm{Con} \;\rightarrow\; H \vdash \mathbf{t} = \mathbf{f}.$$

Both results have standard corollaries: if H is consistent, then neither G nor Con is derivable from H.

2 The formal system

Trees

The ground data are finite binary trees:

$$a, b \;::=\; O \;\mid\; \langle a, b\rangle$$

where O is the leaf and $\langle a, b\rangle$ is the node. We define:

$$\mathbf{t} \;=\; O, \qquad \mathbf{f} \;=\; \langle O, O\rangle.$$

Here $\mathbf{t}$ represents equality (or "yes") and $\mathbf{f}$ represents inequality (or "no"). We write $\overline{n}$ for the unary encoding of $n \in \mathbb{N}$: $\overline{0} = O$, $\overline{n+1} = \langle O, \overline{n}\rangle$.

Terms and functors

Terms are built from trees, variables, and functor applications. We use t, u, v for terms, a, b for trees. The system has countably many variables $x_0, x_1, x_2, \ldots$. There are unary functors ($f, g : \mathrm{Fun}_1$) and binary functors ($h : \mathrm{Fun}_2$).

f, g	$::=$	I	identity
	$\mid$	π_1	first projection
	$\mid$	π_2	second projection
	$\mid$	$f \circ g$	composition
	$\mid$	$h[f, g]$	binary composition: $t \mapsto h(f(t), g(t))$
	$\mid$	$\mathrm{R}(z, s)$	tree recursion with base z and step s
	$\mid$	K_c	constant c
h	$::=$	P	pairing: $(t, u) \mapsto \langle t, u \rangle$
	$\mid$	C	left projection: $(t, u) \mapsto t$
	$\mid$	$\mathrm{Lf}(f)$	lift: $(t, u) \mapsto f(t)$
	$\mid$	$\mathrm{Ps}(f, h)$	post-compose: $(t, u) \mapsto f(h(t, u))$
	$\mid$	$\mathrm{Fn}(h_1, h_2, h)$	fan: $(t, u) \mapsto h(h_1(t, u),\ h_2(t, u))$
	$\mid$	If	conditional
	$\mid$	TreeEq	tree equality test

Terms:

$$t, u \ ::= \ O \ \mid \ x_i \ \mid \ f(t) \ \mid \ h(t, u)$$

An *equation* is a pair $t = u$ of terms.

Embedding trees into terms

Every tree a determines a closed term $\hat{a}$:

$$\hat{O} = O, \qquad \widehat{\langle a, b \rangle} = \langle \hat{a}, \hat{b} \rangle.$$

(In Agda: `reify`.) Conversely, every closed term reduces to $\hat{a}$ for a unique a.

Derivation rules

A derivation is built from the rules below. Every rule may use a hypothesis H; we write $H \vdash t = u$.

Computation axioms.

Identity:	$\mathrm{I}(t) = t$
First projection:	$\pi_1(\langle t, u \rangle) = t$
Second projection:	$\pi_2(\langle t, u \rangle) = u$
Constant:	$\mathrm{C}(t, u) = t$
Composition:	$(f \circ g)(t) = f(g(t))$
Binary composition:	$h[f, g](t) = h(f(t),\ g(t))$
Lift:	$\mathrm{Lf}(f)(t, u) = f(t)$
Post-compose:	$\mathrm{Ps}(f, h)(t, u) = f(h(t, u))$
Fan:	$\mathrm{Fn}(h_1, h_2, h)(t, u) = h(h_1(t, u),\ h_2(t, u))$
Constant functor:	$\mathrm{K}_c(t) = c$

Tree recursion.

$$\begin{aligned} &\text{Base:} && \mathrm{R}(z,s)(O) = z \\ &\text{Step:} && \mathrm{R}(z,s)(\langle t,u\rangle) = s\big(\langle t,u\rangle,\ \langle \mathrm{R}(z,s)(t), \mathrm{R}(z,s)(u)\rangle\big) \end{aligned}$$

Conditional.

$$\begin{aligned} &\text{True branch:} && \mathrm{If}(\mathbf{t},\ \langle t,u\rangle) = t \\ &\text{False branch:} && \mathrm{If}(\langle t_1,t_2\rangle,\ \langle u_1,u_2\rangle) = u_2 \end{aligned}$$

More generally, If branches on whether its first argument is O (leaf) or a pair (node).

Tree equality.

$$\begin{aligned} &\mathrm{TreeEq}(O,O) && = \mathbf{t} \\ &\mathrm{TreeEq}(O,\langle t,u\rangle) && = \mathbf{f} \\ &\mathrm{TreeEq}(\langle t,u\rangle,O) && = \mathbf{f} \\ &\mathrm{TreeEq}(\langle t_1,t_2\rangle,\langle u_1,u_2\rangle) && = \mathrm{If}\big(\mathrm{TreeEq}(t_1,u_1),\ \langle \mathrm{TreeEq}(t_2,u_2),\mathbf{f}\rangle\big) \end{aligned}$$

TreeEq returns $\mathbf{t}$ for equal trees and $\mathbf{f}$ for unequal. The last clause says: if the left children differ, return $\mathbf{f}$ immediately; otherwise, recurse on the right children.

In particular $\mathrm{TreeEq}(t,t) = \mathbf{t}$ for all t, proved by **Schema F** (`treeEqSelf` in `GodelII.agda`).

Structural rules.

$$\frac{}{t = t}\ (\mathrm{refl}) \qquad \frac{t = u}{u = t}\ (\mathrm{sym}) \qquad \frac{t = u \qquad u = v}{t = v}\ (\mathrm{trans})$$

$$\frac{t = u}{f(t) = f(u)}\ (\mathrm{cong}_1) \qquad \frac{t = u}{h(t,v) = h(u,v)}\ (\mathrm{cong}_L) \qquad \frac{t = u}{h(v,t) = h(v,u)}\ (\mathrm{cong}_R)$$

Substitution.

$$\frac{H \vdash t = u}{H \vdash t[c/x_i] = u[c/x_i]}\ (\mathrm{inst})$$

Hypothesis.

$$\frac{}{H \vdash H}\ (\mathrm{hyp})$$

Schema F (tree induction). If f and g are unary functors, z a term, s a binary functor, and we have derivations of:

$$\begin{aligned} &f(O) = z, && f(\langle x_0,x_1\rangle) = s(\langle x_0,x_1\rangle,\ \langle f(x_0), f(x_1)\rangle), \\ &g(O) = z, && g(\langle x_0,x_1\rangle) = s(\langle x_0,x_1\rangle,\ \langle g(x_0), g(x_1)\rangle), \end{aligned}$$

then $f(x_0) = g(x_0)$.

Schema F expresses the uniqueness of tree recursion: if f and g satisfy the same recurrence (same base z, same step s), they agree on all inputs. Restricted to unary trees $\overline{n}$, this gives ordinary mathematical induction.

Consistency

The system is *consistent* (relative to hypothesis H) if there is no derivation of $\mathbf{t} = \mathbf{f}$ from H.

3 Key components

3.1 Proof encoding (thFunTFor, Theorem 14)

thFunTFor is a unary functor (defined as $\mathrm{R}(O, \mathrm{thFunStep})$) that maps proof-encoding trees to the code of the equation they prove. The step function thFunStep is a 26-branch dispatch handling all derivation rules.

Every term t is encoded as a tree $\ulcorner t \urcorner$, and every equation $t = u$ as $\ulcorner t = u \urcorner = \langle \ulcorner t \urcorner, \ulcorner u \urcorner \rangle$. Every functor is similarly encoded.

Theorem 14 (thm14E): if $H \vdash t = u$, then there exist trees p_a, p_b such that:

1. $\mathrm{thFun}(\langle p_a, p_b \rangle) = \ulcorner t = u \urcorner$ (meta-level correctness),
2. $\vdash \mathrm{thFunTFor}(\langle \widehat{p_a}, \widehat{p_b} \rangle) = \widehat{\ulcorner t = u \urcorner}$ (object-level correctness, polymorphic in hypothesis).

The proof is by induction on the derivation, with 26 cases (1402 lines in `Thm14E.agda`).

Component (2) is *polymorphic*: the derivation uses only computation axioms and structural rules, never the hypothesis rule. This is crucial for Gödel II.

3.2 Coded substitution (substTFor)

substTFor is a unary functor implementing coded substitution: given a coded term $\ulcorner t \urcorner$, it replaces every occurrence of variable x_n by a replacement term r.

These two parameters (r and n) are represented by free variables in substTFor. To avoid collision with the variables x_0, x_1 used in **Schema F** and in the template equation, the formalization places them at high indices:

- x_{11} carries the replacement r,
- x_{12} carries the target index $\overline{n}$.

Closing these variables gives the fully applied version:

$$\mathrm{closedSubstTFor}(r, \overline{n}) \;=\; \mathrm{substTFor}[r/x_{11},\ \overline{n}/x_{12}].$$

Correctness: for any term t, $\mathrm{closedSubstTFor}(r, \overline{n})$ applied to $\widehat{\ulcorner t \urcorner}$ equals $\ulcorner \widehat{t[r/x_n]} \urcorner$.

3.3 $\mathrm{TreeEq}(t, t) = \mathbf{t}$ via Schema F

$\mathrm{TreeEq}(t, t) = \mathbf{t}$ for all t is proved using **Schema F**. Define $f = \mathrm{TreeEq}[\mathrm{I}, \mathrm{I}]$ (so $f(t) = \mathrm{TreeEq}(t, t)$) and $g = \mathrm{K}_{\mathbf{t}}$. Both satisfy the same recursion:

- f: by the computation axioms for TreeEq and the recursive structure of TreeEq on $\langle t_1, t_2 \rangle$;
- g: by $\mathrm{K}_{\mathbf{t}}$; the step reduces to $\mathrm{If}(\mathbf{t}, \langle \mathbf{t}, \mathbf{f} \rangle) = \mathbf{t}$ since g returns $\mathbf{t}$ on all inputs.

Schema F gives $f(t) = g(t)$, i.e., $\mathrm{TreeEq}(t, t) = \mathbf{t}$.

4 The Gödel sentence and diagonal construction

Template

The Gödel sentence is built from a *template equation* with several free variables:

- x_0: will receive a proof encoding (the "proof slot");
- x_1: will receive a coded term (the "diagonal slot");
- x_{11}, x_{12}: the replacement and target parameters of substTFor (closed later).

The template is:

$$\text{templateEqn} \;=\; \big(\text{TreeEq}(\text{thFunTFor}(x_0),\ \text{substTFor}(x_1)) = \mathbf{f}\big).$$

It says: "the equation encoded by the proof in x_0 is not equal to the result of substituting via x_1." Leaving x_{11}, x_{12} free at this stage keeps the tree code of the template small.

Diagonalization

Let $N = \ulcorner\text{templateEqn}\urcorner$ be the tree encoding of the template. The Gödel sentence is obtained by substituting $\hat{N}$ (the numeral for N) into the diagonal slot:

$$G \;=\; \text{templateEqn}[\hat{N}/x_1].$$

After this substitution, G says (informally): "for any proof tree in x_0, the equation it encodes is not the result of substituting N into the template" — i.e., "no proof tree proves G."

Abstract definitions (cached)

The following are computed once in `SubstTForPrecomp.agda` and cached:

- N: tree code of the template
- $\hat{N}$: the numeral $\widehat{N}$
- $G = \text{templateEqn}[\hat{N}/x_1]$
- $\ulcorner G\urcorner$: tree encoding of G
- cstf $=$ closedSubstTFor$(\widehat{\hat{N}},\ \overline{1})$: the substitution functor closed with replacement $=$ code of the template numeral and target $=$ variable x_1

Heavy computation (one-time)

`SubstTForPrecomp.agda` contains one step requiring ∼108 seconds (computed once, cached in `.agdai`): proving that after instantiating x_{11} and x_{12}, the open substTFor becomes cstf. This requires Agda to traverse $\widehat{\hat{N}}$ (∼400K term nodes) with substitution acting as identity on the closed subterm.

Remark 3. *The mathematical argument does not require this traversal. A general "substitution is identity on closed terms" lemma would eliminate it.*

5 Proof of Gödel I

Theorem 4 (Gödel I, constructive form)**.** *For any hypothesis equation H: $H \vdash G \;\to\; H \vdash \mathbf{t} = \mathbf{f}$.*

Corollary 5 (Gödel I, standard form — `godelI` in `ProofEAny.agda`)**.** *Consistent$(H) \;\to\; H \vdash G \;\to\; \bot$.*

Both forms remove the ProofE(H) assumption from the original proof. The key enabler is mkProofEAny, which constructs a proof encoding for any equation using the trans-from-two-refls technique.

The proof in `GodelII.agda` (0.2 seconds, 168 lines) follows Rose's Theorem 18 [**?**]. We assume $H \vdash G$ and derive $H \vdash \mathbf{t} = \mathbf{f}$:

1. **Assume** $H \vdash G$.
2. **Theorem 14** gives a proof encoding e with
$$H \vdash \text{thFunTFor}(e) = \widehat{\ulcorner G \urcorner}.$$
3. **diagFTarget** (precomputed): $H \vdash \text{cstf}(\hat{N}) = \widehat{\ulcorner G \urcorner}$.
4. **Chain** steps 2 and 3 through $\widehat{\ulcorner G \urcorner}$ as middle term (trans + sym):
$$H \vdash \text{thFunTFor}(e) = \text{cstf}(\hat{N}).$$
The term $\widehat{\ulcorner G \urcorner}$ appears only as the middle of a transitivity chain — it is never touched by inst.
5. **Instantiate** G with e (inst on x_0):
$$H \vdash \text{TreeEq}(\text{thFunTFor}(e),\ \text{substTFor}(\hat{N})) = \mathbf{f}.$$
6. **Replace** thFunTFor(e) with cstf$(\hat{N})$ using cong$_L$ and the chain from step 4:
$$H \vdash \text{TreeEq}(\text{cstf}(\hat{N}),\ \text{substTFor}(\hat{N})) = \mathbf{f}.$$
7. **Instantiate** x_{11} and x_{12} (inst, using the cached computation from Section 4): the open substTFor$(\hat{N})$ becomes cstf$(\hat{N})$:
$$H \vdash \text{TreeEq}(\text{cstf}(\hat{N}),\ \text{cstf}(\hat{N})) = \mathbf{f}.$$
8. **Reflexivity** (TreeEq$(t, t) = \mathbf{t}$ via **Schema F**):
$$H \vdash \text{TreeEq}(\text{cstf}(\hat{N}),\ \text{cstf}(\hat{N})) = \mathbf{t}.$$
9. **Contradiction**: by trans, $H \vdash \mathbf{t} = \mathbf{f}$, violating consistency.

6 Proof of Gödel II

The internal consistency statement

The code of the contradiction $\mathbf{t} = \mathbf{f}$ is

$$\ulcorner \bot \urcorner \;=\; \ulcorner \mathbf{t} = \mathbf{f} \urcorner \;=\; \langle \ulcorner O \urcorner, \ulcorner \langle O, O \rangle \urcorner \rangle.$$

Definition 6. *The* internal consistency statement *is the equation*

$$\mathrm{Con} \;\equiv\; \mathrm{TreeEq}\big(\mathrm{thFunTFor}(x),\, \widehat{\ulcorner \bot \urcorner}\big) \;=\; \mathbf{f}$$

where x is a free variable, implicitly universally quantified. Since $\mathrm{TreeEq}(a, b) = \mathbf{f}$ *means $a \neq b$,* Con *says:* for every proof tree x, the equation encoded by x is not $\mathbf{t} = \mathbf{f}$ — *i.e., the system does not prove* $\mathbf{t} = \mathbf{f}$.

When the hypothesis is a trivially true equation such as $\mathbf{t} = \mathbf{t}$, this corresponds exactly to Guard's Theorem 14: "$\mathrm{th}(y) \neq$ "$0 = 1$" is valid but unprovable in BRA" [**?**].

Statement

Theorem 7 (Gödel II, constructive form — `conToBot` in `GodelIIFull.agda`)**.** *For any hypothesis equation H:*

$$H \vdash \mathrm{Con} \;\to\; H \vdash \mathbf{t} = \mathbf{f}.$$

That is: if the system derives Con, then it derives $\mathbf{t} = \mathbf{f}$. This is the direct constructive content — no consistency assumption, no Markov's principle.

Corollary 8 (Gödel II, standard form — `godelII` in `GodelIIFull.agda`)**.** *For any hypothesis equation H:*

$$\mathit{Consistent}(H) \;\to\; H \vdash \mathrm{Con} \;\to\; \bot.$$

Remark 9. *The constructive form (Theorem* **??***) is stronger than the standard form (Corollary* **??***). Classically they are equivalent by contraposition. Constructively, our proof directly produces the derivation $H \vdash \mathbf{t} = \mathbf{f}$ from $H \vdash \mathrm{Con}$ without any classical reasoning: the consistency assumption is applied only at the final step to obtain $\bot$.*

Proof

The proof has three components.

Step 1: Pre-compute the encoding of the Gödel I argument.
Specialize the Gödel I proof to the hypothesis $H_0 = G$ (the Gödel sentence itself). With this hypothesis, ruleHyp gives $G \vdash G$, and the Gödel I argument (Theorem **??**) produces a derivation:

$$d_\bot : G \vdash \mathbf{t} = \mathbf{f}.$$

Apply Theorem 14 (`thm14E`) to $d_\bot$ to obtain the proof encoding $\mathit{enc}_\bot$ and the correctness proof:

$$\mathit{corrBot} : \mathrm{thFunTFor}(\mathit{enc}_\bot) = \widehat{\ulcorner \bot \urcorner}.$$

Crucially, *corrBot* is *polymorphic in the hypothesis*: it is a derivation using only computation axioms and structural rules, not the hypothesis rule. Therefore it holds in any hypothesis context, not just under G.

The term $enc_\perp$ is a specific closed term computed at the Agda meta-level. It encodes the entire Gödel I proof: the diagonal (`diagFTarget`), tree equality self-comparison (`treeEqSelf`), variable instantiation, and all 26 case verifications through `thm14E`.

Step 2: Instantiate the consistency hypothesis.
Given $H \vdash \mathrm{Con}$, apply variable substitution (`ruleInst`) with $x := enc_\perp$:

$$H \vdash \mathrm{TreeEq}(\mathrm{thFunTFor}(enc_\perp),\ \widehat{\ulcorner \perp \urcorner}) = \mathbf{f}.$$

Step 3: Derive the contradiction.
From *corrBot* (polymorphic, instantiated at H):

$$H \vdash \mathrm{thFunTFor}(enc_\perp) = \widehat{\ulcorner \perp \urcorner}.$$

Replace $\mathrm{thFunTFor}(enc_\perp)$ with $\widehat{\ulcorner \perp \urcorner}$ by congruence:

$$H \vdash \mathrm{TreeEq}(\widehat{\ulcorner \perp \urcorner},\ \widehat{\ulcorner \perp \urcorner}) = \mathbf{f}.$$

But $\mathrm{TreeEq}(t, t) = \mathbf{t}$ for all t (by **Schema F**):

$$H \vdash \mathrm{TreeEq}(\widehat{\ulcorner \perp \urcorner},\ \widehat{\ulcorner \perp \urcorner}) = \mathbf{t}.$$

By transitivity: $H \vdash \mathbf{t} = \mathbf{f}$, contradicting $\mathit{Consistent}(H)$. □

7 Discussion

Comparison with Guard and Rose

Guard [**?**] proves Gödel II (his Theorem 14) by explicitly constructing a functor $h(x)$ within BRA such that $\mathrm{th}(h(x)) =$ "$0 = 1$" whenever $\mathrm{th}(x) = j$ (the Gödel sentence code). This uses his Theorem 13 (internalization: $f(x) = y \supset \mathrm{th}(D_f(x)) =$ "$fx = y$").

Rose [**?**] proves Gödel II (his Theorem 18) by applying his Theorem 16 (internalized Schema F) to the function $a(\mathrm{se}(\mathrm{nu}(N), N), \mathrm{th}(y))$, using the consistency hypothesis $\mathrm{th}(x) \neq e(\mathrm{th}(z))$. Cook [**?**] points out a possible error in Rose's Theorem 16 (p. 134 of [**?**]): the function $f(s(x))$ is neither identically zero nor identically non-zero, which invalidates the argument at that step.

Our proof takes a different route: rather than constructing h as an object-level functor or applying internalized Schema F, we *pre-compute* the encoding $enc_\perp$ at the Agda meta-level from the concrete Gödel I derivation. The polymorphicity of Theorem 14's correctness proof (*corrBot* holds in any hypothesis context) is the key property that makes this work, bypassing Rose's problematic Theorem 16 entirely.

The role of Theorem 14

The entire non-trivial content of the proof resides in Theorem 14 (`thm14E`). The 26 case files verify that thFunTFor correctly tracks each derivation rule. The Gödel II proof applies Theorem 14 to the specific derivation $d_\perp$ (the Gödel I argument with hypothesis G), obtaining an encoding whose thFunTFor image is the contradiction code. The instantiation of Con at this encoding then produces the contradiction.

Connection with Cook's PV

Cook [**?**] suggests (Section 8) that a natural formalization of feasibly constructive proofs should take a *programming approach*, where "proving $f(x) = g(x)$ amounts to proving the equivalence of two programs." The Guard/Rose system realizes this vision literally: proving $f(x_0) = g(x_0)$ via **Schema F** *is* proving that two recursive programs (the functors f and g) compute the same function. Every derivation in the system is a chain of equational rewrites on functor expressions — no quantifiers, no propositional connectives, only computation axioms and the uniqueness-of-recursion principle.

8 Correspondence with Rose

Rose [?]	**This formalization**
$th(y)$	thFunTFor
$se(x, y, z)$ / $st(x, y, z)$	substTFor / closedSubstTFor
$nu(x)$	implicit via $\hat{N}$
N (template code)	templateCode
Schema F	ruleF (uniqueness of recursion)
Theorem 10	thm14E (ProofE)
Theorem 14	thm14E
Theorems 15–16	Bypassed (see Discussion); cf. [**?**] on Thm 16
Theorem 17	diagFTarget
Theorem 18	`godelII` in `GodelII.agda` (= Gödel I)
—	`godelII` in `GodelIIFull.agda` (= Gödel II)

9 File structure

File	Time	Role
Base.agda	< 1s	Natural numbers, equality, sigma types
Term.agda	< 1s	Terms, functors, coding functions
Step.agda	< 1s	Derivation system (26 rules + **Schema F**)
StepReduce.agda	< 1s	Derived reduction lemmas
ThFun.agda	< 1s	Metalevel proof evaluator
ThFunCorrect.agda	< 1s	Metalevel correctness
ThFunTFor.agda	< 1s	Object-level proof evaluator (26 branches)
ThFunTForDefs.agda	< 1s	Definitions for thFunTFor cases
ThFunTForCases0–3.agda	< 1s	Per-tag case proofs
ThFunTForCorrectDefs.agda	< 1s	Correctness infrastructure
Thm14E.agda	< 1s	Theorem 14 (1402 lines, all 26 cases)
SubstTFor.agda	< 1s	Coded substitution functor
SubstCorrect.agda	< 1s	Substitution correctness
SubstTForCorrect.agda	< 1s	closedSubstTFor correctness
SubstTForPrecomp.agda	108s	**Heavy**: precomputed abstractions
SubstTForPrecomp2.agda	< 1s	Assembly from opaque pieces
NdDispatchProofs.agda	< 1s	Node-dispatch case proofs
PairPassthrough.agda	< 1s	Pair passthrough lemmas
PairPassthroughAll.agda	< 1s	Extended passthrough
IntermediatePassthrough.agda	< 1s	Intermediate passthrough
ExtractorRed.agda	< 1s	Extractor reduction lemmas
GodelII.agda	< 1s	Gödel I (168 lines) + treeEqSelf
ProofEAny.agda	< 1s	mkProofEAny + strengthened Gödel I
GodelIIFull.agda	< 1s	**Gödel II** ($\sim$90 lines)
CookPV.agda	< 1s	Boolean ops, De Morgan, excluded middle
NoPChecker.agda	< 1s	No proof-checker (corollary of Gödel II)

All results compile with Agda 2.9.0 using `--safe --without-K --exact-split`. No postulates. The 108-second computation is cached and not repeated.

References

[1] J. R. Guard, *Lecture Notes on Recursive Arithmetic*, Princeton University, 1963.

[2] H. E. Rose, "On the consistency and undecidability of recursive arithmetic," *Zeitschr. f. math. Logik und Grundlagen d. Math.*, vol. 7, pp. 124–135, 1961.

[3] S. A. Cook, "Feasibly constructive proofs and the propositional calculus," *Proceedings of the Seventh Annual ACM Symposium on Theory of Computing*, pp. 83–97, 1975.